Voor jouw apotheek

Svelte Fundamentals

Module – Binding Forms

Peter Kassenaar – info@kassenaar.com

eduvision
opleiding & training

Binding to form fields

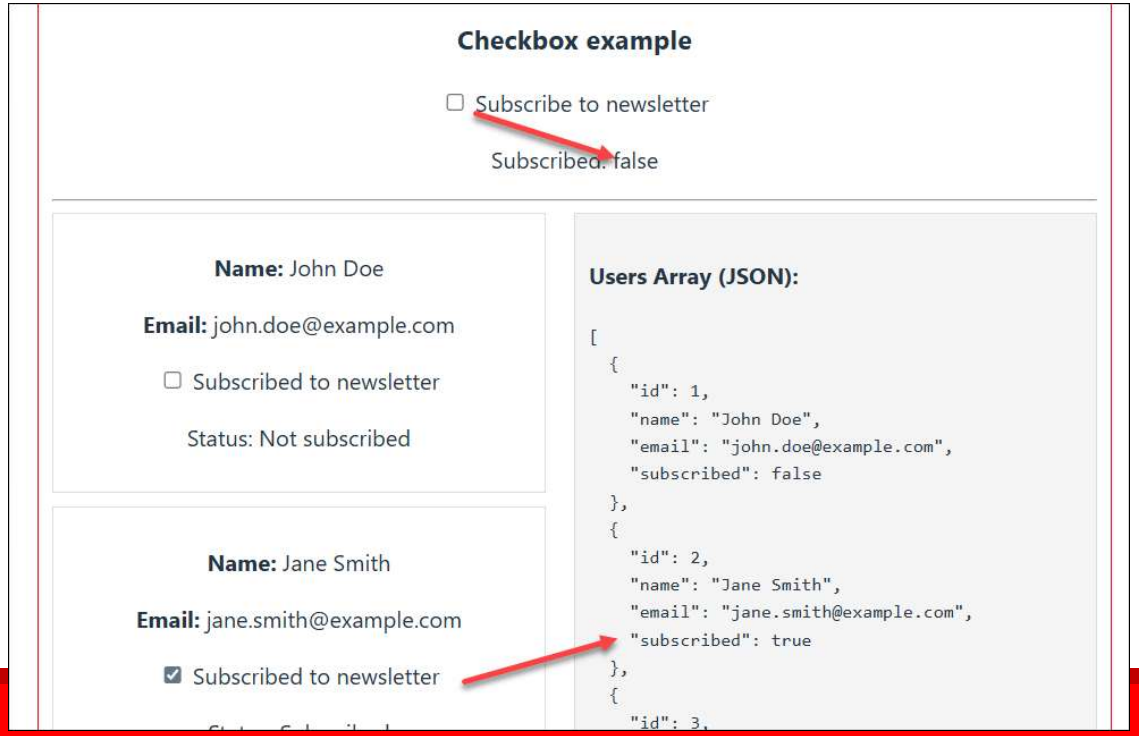
Binding values to well known fields like `text`, `checkbox`, `radio` and so on

Binding data to form fields

- All kinds of binding use the `bind:` directive
- Binding to a text field:
  - `<input type="text" bind:value={someVariable} />`
  - two-way binding by default
  - Variable is updated when the user types
  - DOM is updated when variable changes

Binding to a checkbox

- Checkboxes use `bind:checked`, not `value`
  - `let subscribed = false;`
  - `<input type="checkbox" bind:checked={subscribed}/>`
- Can also be used with objects, not just variables

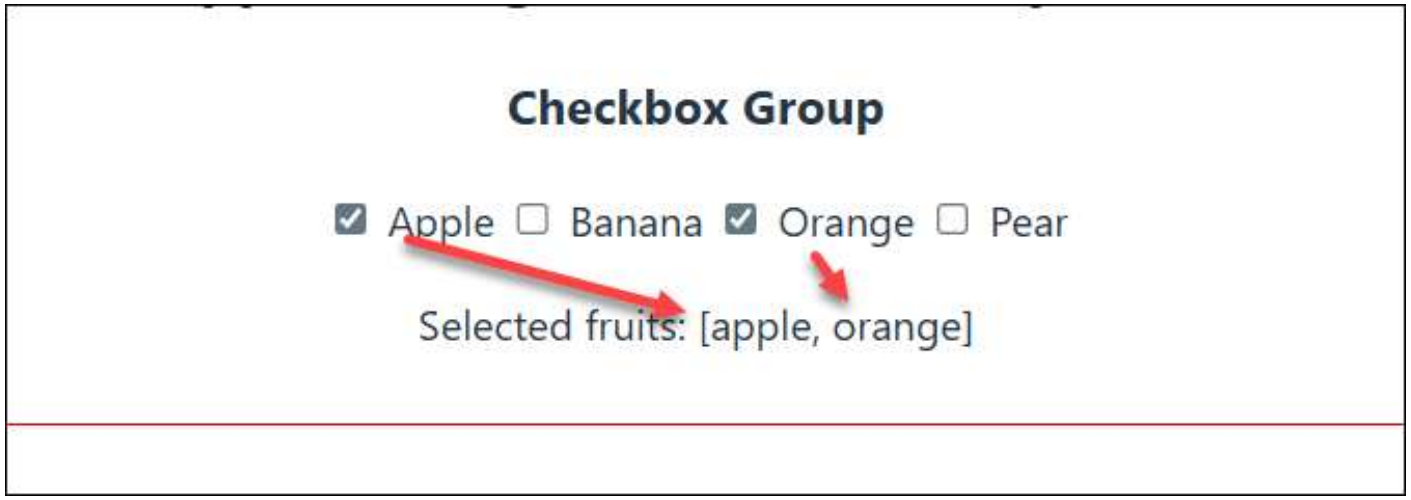


Binding to radio buttons

- Radio buttons use `bind:group` – not `value`
  - `let color = 'red';`
  - `<input type="radio" bind:group={color} value="red" />`
- Again, this can also be used with objects

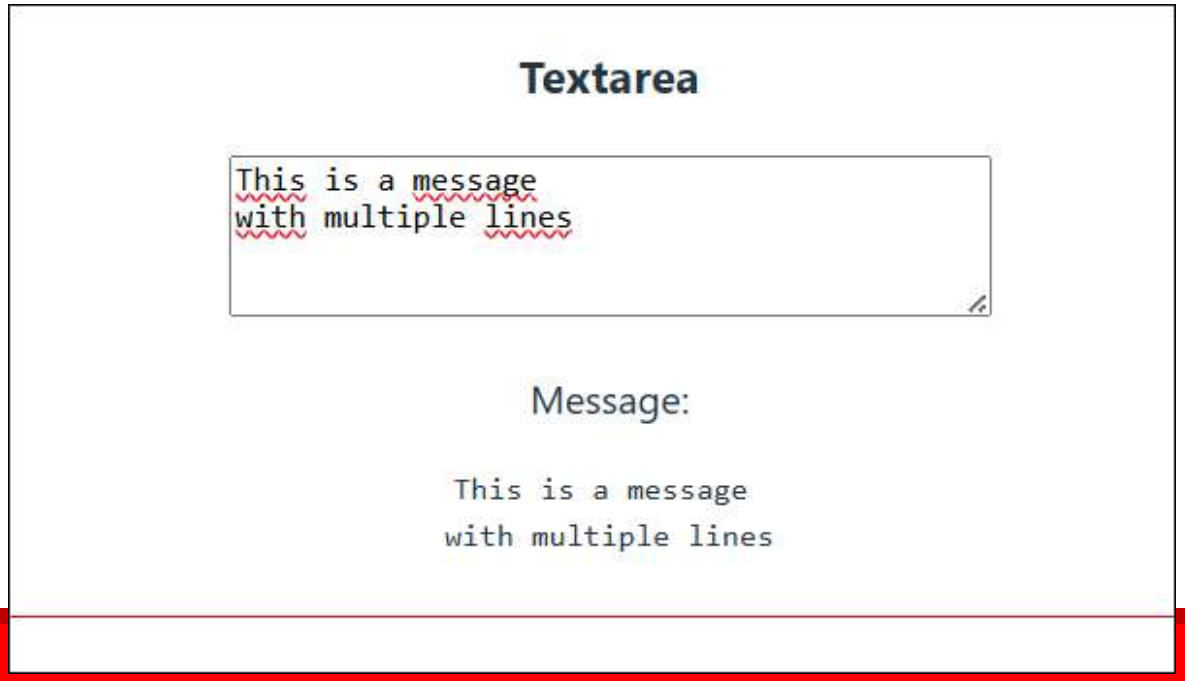
Extra: multiple checkboxes in array

- Also: `bind:group` removes/adds items to an array
  - `let fruits: string[] = […];`
  - `<input type="checkbox" bind:group={fruits} value="apple" />`
  - `<p>Selected fruits: {fruits.join(', ')}</p>`



Binding to `<textarea>`

- Binding to a `<textarea>` is the same as binding to a single line textfield
  - Note: same `bind:value` as text inputs
  - Works for `<textarea>` as well




```
<textarea
        rows="4"
        cols="40"
        bind:value={msg}
        placeholder="…"
></textarea>
```

Two-way binding behind the scenes

- Actually, it is not magic. Svelte two-way binding is just shorthand notation for
  - `<input bind:value={name} /> // shorthand`
  - `<input value={name} on:input={(e) => name = e.target.value}/>`

Binding to a dropdown, `<select>` and `<option>`

- Binding to a dropdown is (also) the same as binding to a text input field.
  - Use `bind:value={someVariable | someProperty}`
  - The UI is updated when `someVariable | someProperty` changes, based on the selected value of the dropdown.

Example: using `<select>` and `<option>`

```
const countries = [
    { code: 'NL', name: 'Netherlands' },
    { code: 'DE', name: 'Germany' },
    { code: 'FR', name: 'France' },
    { code: 'ES', name: 'Spain' }
];
```

```
<p>Selected country code:
<strong>{selectedCountry}</strong></p>
<select bind:value={selectedCountry}>
    {#each countries as country}
        <option value={country.code}>
            {country.name}
        </option>
    {/each}
</select>
```

Using complete Objects

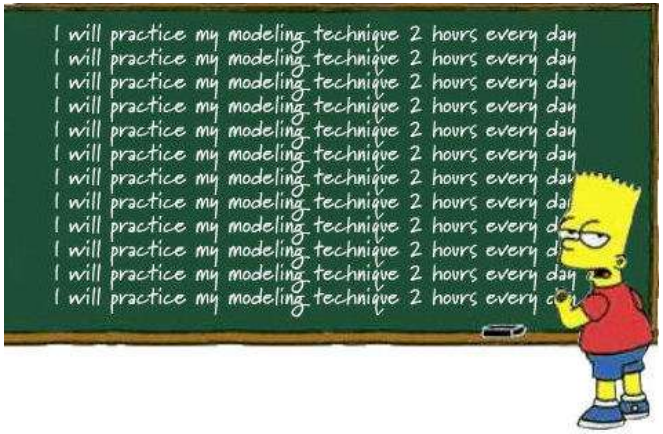
- Instead of using object properties as bindable props, we can also use complete objects
  - For instance:

```
let selectedCountryObject: Country;
```

```
<select bind:value={selectedCountryObject}>
    {#each countries as country}
        <option value={country}>
            {country.name}
        </option>
    {/each}
</select>
```

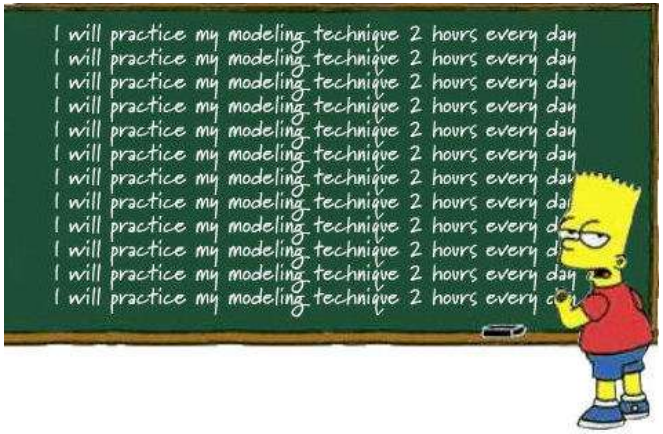
Mini-workshop

- Create a component with a small form
- Add a dropdown Favorite programming language
  - `TypeScript`
  - `Rust`
  - `C#`
  - `Java`
  - `BASIC`
- Add a button to ‘submit’ the selected value:
- Show the value in an `alert` box or a `console.log()`



Main Workshop

- Build a more extensive `ProfileForm` Component.
  - `firstName`, `lastName` – textbox, string
  - `Available for freelance work` – checkbox, boolean
  - `Experience level`: `junior`, `medior`, `senior` – radio buttons, variable `level`
  - `Skills`: `HTML, CSS, JavaScript, Svelte`, … - checkbox group, `skills, string []`
  - `Short bio` – textarea
  - Country: `NL, DE, FR, BE, UK` – dropdown
- Create 2 columns:
  - Left column: form
  - Right colum: summary of fields
- Buttons: `Submit` and `Clear`



Example form from workshop

Your Profile (from workshop):

First name Peter

Last name Kassenaar

☑ Available for freelance work?

Experience level

○ Junior ○ Medior ◉ Senior

Skills

☑ HTML ☐ CSS ☐ JavaScript ☐ TypeScript ☑ Svelte

Bio

I am an experienced developer, blablabla...

Submit Reset

Live profile preview

Name: Peter Kassenaar

Available: true

Level: senior

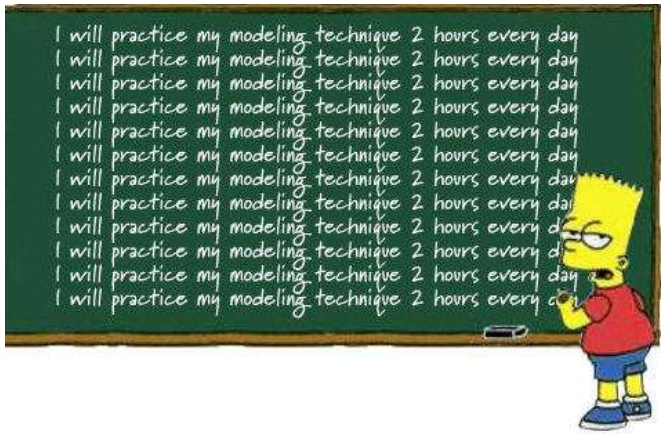
Skills: HTML, Svelte

Bio:

I am an experienced developer, blablabla...

Workshop extensions

- Show the number of selected skills
- Show `Yes | No` instead of `true | false`
- Add placeholders to text fields
- Use a dropdown instead of radio button group
- Add some default values
- Add Character counter to Bio
- Add simple validation:
  - Name is required
  - Bio must be at least 40 characters



(using AI? Make sure to tell you use Svelte 5(!))

Checkpoint

- You know about all kinds of binding values to form fields
- You can create complex forms using Svelte bindings
- You can submit and reset forms.